

Knowledge Graphs and Large Language Models

A Complementary Architecture for Digital-Asset Threat Intelligence

Conceptual foundations for the RISC-LLM-TI platform — thesis material

1. The Problem, and Why It Resists a Pure-LLM Solution

Institutions issuing and custodying tokenised assets on public blockchains face a threat environment that is **shared, composable, and continuously evolving**. A tokenised deposit on a layer-2 network inherits the risk of the bridge it depends on, the consensus of the chain beneath it, and the security of every protocol it touches. Threat intelligence about this environment arrives as unstructured prose: incident post-mortems, auditor disclosures, CVE advisories, on-chain forensics, and analyst notes. The institutional question is not “what does this one report say” but “given everything known across all of these sources, what is my exposure, why, and what must I remediate.”

A large language model alone cannot answer this durably. It can read any single document fluently, but it has **no persistent memory across documents**, no auditable chain of evidence, and no guarantee that it has not silently omitted a critical fact. Asked the same question twice it may answer differently. For a regulated institution making capital and custody decisions, an answer that cannot be traced to its evidence, reproduced, or defended to an auditor is not usable. This is the gap the present work addresses.

2. Ontology and Knowledge Graph: Definitions and Their Relationship

Two artefacts sit at the centre of the architecture, and the distinction between them is frequently muddled. Stating it precisely is essential.

2.1 The ontology is the schema of what can exist

The ontology is an explicit, machine-checkable model of a domain: the **types of entities** that may exist (ThreatActor, Vault, Vulnerability, KeyShare, Control), the **types of relationship** permitted between them (a ThreatActor may EXPLOIT a Vulnerability; a Vulnerability may LEAK a KeyShare), and the **constraints** that govern them (only an actor can exploit; a key share belongs to exactly one vault). The ontology encodes human domain expertise in a form a machine can enforce. It contains no facts about any particular incident — it is the empty form, the contract, the grammar of the domain.

2.2 The knowledge graph is the populated instance

The knowledge graph is the schema filled in with real, extracted facts that conform to it:

Dinosaurus (a ThreatActor) EXPLOITS the GG20 proof omission (a Vulnerability), which LEAKED the TSS key share (a KeyShare) of the Asgard Vault (a Vault).

Each node is a typed entity; each edge is a typed, directed relationship; and — critically in this work — every node and edge carries **provenance**: the source document and the exact sentence that supports it,

together with a confidence value. The graph is the filled-in form, across thousands of such facts drawn from many documents.

2.3 The relationship between them is governance

The ontology **constrains** what the graph may assert. During construction, any candidate fact whose types violate the schema is rejected before it can enter the graph. This is precisely what converts an otherwise free-form extraction into an **auditable, typed, validated** knowledge base. The ontology is therefore not documentation that sits beside the graph; it is the active validation boundary through which every fact must pass. This relationship is the first half of the answer to “why not simply ask a language model.”

Dimension	Ontology	Knowledge graph
What it is	Schema / vocabulary / rules	Populated instance of facts
Contains	Types, relations, constraints	Specific entities and relationships
Analogy	The empty form	The filled-in form
Changes	Rarely (per domain)	Continuously (per document ingested)
Role	Governs what may be asserted	Records what has been asserted
Built by	Expert design + LLM induction	LLM extraction, schema-validated

3. Why the Knowledge Graph Is Necessary, Not Decorative

A common objection is that the graph is a presentation layer — a way to draw what the documents already say. If that were all it did, the objection would be fatal. The graph earns its place through three properties an LLM cannot provide, each of which maps to a concrete institutional need.

3.1 Persistent, accumulating institutional memory

A human analyst who reads an incident post-mortem holds the picture in their head; when they leave, it is gone. The graph persists that understanding as structured, queryable knowledge, and — decisively — it **accumulates**. Every new incident ingested adds to a single growing body of organisational understanding. The model is invoked once per document to extract; the resulting knowledge is reused indefinitely and compounds. This is the “use the expensive model once, reuse the knowledge forever” economics, and it is something an LLM, which forgets every conversation, structurally cannot offer.

3.2 Cross-source inference: making the invisible visible

This is the core of the novelty claim. Any single report conveys only what its author knew. The graph derives knowledge that **no single source states**, by composing facts across documents. Three representative patterns:

- **Shared infrastructure:** the same attacker wallet appears in two unrelated incident reports. Neither report mentions the other; the graph reveals the link.
- **Control gaps:** a technique used against Protocol A has no mitigating control recorded for your Protocol B. The exposure exists in the join of two documents, in neither alone.
- **Transitive attack paths:** a three-hop path from a known actor to your asset, through a shared bridge, exists in no document because no document covered all three hops.

This derived knowledge is the “new, actionable intelligence” the platform exists to produce. It is computed by deterministic graph algorithms over the assembled facts, and every derived edge records the derivation that produced it. The graph is a **reasoning substrate**, not a filing cabinet — and reasoning over a typed, connected structure is something a language model operating on flat text cannot do reliably or explainably.

3.3 Explainable, quantified exposure for public-chain assets

For an institution running tokenised assets on public blockchains, the attack surface is shared and transitive: your asset depends on a bridge another protocol’s exploit compromised, on a chain whose reorganisation risk you inherit. A graph is the natural representation of this transitive dependency structure; a list of CVEs or a spreadsheet of incidents fundamentally cannot represent it. On that structure the platform computes **exposure scores** — deterministically, from graph traversal, with every contributing factor traced to specific edges. The number is reproducible and defensible to an auditor, which is the property that makes it institutionally usable.

4. How LLM and Knowledge Graph Complement Each Other

The architecture is deliberately hybrid because the two technologies have **complementary strengths and opposite weaknesses**. Neither alone is sufficient; together they cover each other’s failure modes.

Capability	Large language model	Knowledge graph
Language understanding	Strong — reads messy prose	None on its own
Structured reasoning	Weak, unreliable	Strong, deterministic
Memory across sources	None	Persistent, accumulating
Explainability	Opaque (“black box”)	Native — every fact traced
Reproducibility	Low (sampling)	Total
Handles ambiguity	Yes	No

The division of labour follows directly from this table. The LLM is used **only where language understanding is required**: converting unstructured documents into candidate typed facts (extraction), and proposing the domain schema (ontology induction). Every output it produces is then validated against the ontology and grounded in source text before it is allowed into the graph. The graph is used

for everything that requires **memory, structure, reasoning, and explanation**: persistence, cross-source inference, exposure scoring, and answering questions. When a user asks a question in natural language, the language model may phrase the response — but the facts and entities in that response are retrieved deterministically from the graph, so the model cannot invent or misname them.

The language model supplies understanding; the knowledge graph supplies memory, reasoning, and proof. Each overcomes the other's defining limitation, and the system is correct only because both are present.

4.1 The “RISC” framing as a governance property

The platform restricts the language model to a small, fixed set of intelligence operations — ingest, extract, normalise, map, link — rather than open-ended generation. This constrained operational surface is described as “RISC-style.” The term is used by analogy, not as a literal reduced-instruction-set processor: the operations are high-level pipeline stages, not single-cycle hardware instructions. Framed honestly, the value is a **governance and auditability property**: the model can only perform an audited set of well-defined transformations, which bounds its failure modes and makes the whole pipeline reviewable. For an institutional security function, a bounded and auditable operational surface is worth more than unbounded cleverness.

5. Thesis Claim

In one sentence, defensible to a committee:

RISC-LLM-TI uses a deliberately constrained large language model to convert unstructured threat intelligence into an ontology-governed knowledge graph that accumulates institutional memory, derives cross-source relationships invisible to any single source, and quantifies digital-asset exposure with full evidential traceability — combining the language model's natural-language understanding with the graph's explainable structured reasoning, and overcoming the limitations of each technology used alone.

The remainder of the thesis substantiates each clause of this claim: the construction pipeline (ontology induction and schema-validated extraction), the inference engine (the cross-source derivations), the exposure model (graph-derived, evidence-linked risk), and the evaluation (extraction fidelity, schema coverage, and the value of derived knowledge).

6. Why Not Simply Fine-Tune or Prompt a Model on the Data?

Two final alternatives must be dismissed explicitly, because reviewers will raise them.

1. **“Fine-tune a model on the corpus.”** Fine-tuning bakes statistical patterns into weights; it does not yield an inspectable, queryable, conflict-aware fact base, and it cannot show the chain of evidence behind an answer or be updated incrementally without retraining. It also provides no defence

against confident omission. Fine-tuning may improve the extractor, but it cannot replace the graph.

2. **“Just put all documents in the context window and ask.”** This scales poorly, provides no persistence between sessions, offers no structured reasoning or exposure computation, and gives no auditable provenance. It reproduces precisely the limitations identified in Section 1.

The knowledge graph is therefore not one design choice among equivalent alternatives; it is the component that supplies the persistence, structure, reasoning, and explainability that the institutional problem demands and that a language model cannot provide. The language model, in turn, supplies the language understanding that makes building the graph from unstructured text tractable in the first place. The necessity of each is the necessity of the pair.